Evaluation

Measure whether it finds the right policies.

For: Policy owners and engineers

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

Testing shows the code works. **Evaluation shows whether it finds the right policies.** Do it before real use, and again after every change of model, prompt, embedding model or search settings.

What to measure

Measure	Meaning	Target to start with
Recall	Of the policies that really needed updating, how many did it flag (Yes or Maybe)?	**≥ 0.95**. Missing one is the costly error
Precision	Of the policies it flagged, how many really needed updating?	≥ 0.5 at first. Each false flag costs a reviewer a few minutes
Candidate recall	Did the truly affected clauses even reach the judge (step 3)?	≥ 0.98. If a clause never reaches the judge, no model can flag it
Maybe rate	Share of flagged policies marked Maybe rather than Yes	Watch it fall as the model improves
Unreadable verdicts	Verdicts with "could not be read"	0

Set your own targets with the policy owners. Write them down before you look at results.

Build the case set

Each case is a **real past change** to a head policy, with the list of policies that people actually had to update afterwards.

```
{
  "name": "POL-007 2025 revision",
  "head_policy": "POL-007",
  "subset": "Safety",
  "new_version_file": "private/POL-007-v4.docx",
  "expected_impacted": ["SAF-112", "SAF-130", "SAF-150"],
  "source": "Change request CR-2025-114; updated policies listed in the closure note"
}
```

Where the answers come from matters more than how many cases you have:

- **Use the record of what was actually updated** (change requests, document control logs, revision histories), not a fresh list someone writes while looking at this system's output.

- **Load the policy library as it was before the change.** Otherwise the affected policies already contain the new wording, and the run finds nothing to flag.
- **Keep a separate hold-out set.** Tune `SEMANTIC_K`, `RERANK_KEEP` and prompts on one set of cases, and report results on a different set you never looked at while tuning. A score on the cases you tuned against tells you little.
- **Include hard cases on purpose:** a renamed defined term, a renumbered section, a removed section, a policy that's affected but never cites the head policy, and a change that affects nothing.
- Start with **10–20 cases**, and grow to 50+ over time. Store them in `eval/private/` and `eval/cases.json`, which git ignores.

Historical records are imperfect: people sometimes missed policies too. When the system flags a policy that the record doesn't list, have the policy owner decide. If it really was affected, fix the case and note it in `source`. This is often the most useful finding.

Run it

```
docker compose exec policy-api python -m app.evaluate /eval/cases.json --min-recall 0.95
```

- By default, Yes **and** Maybe count as found, because both reach a person. `--yes-only` measures how often the model is confident and right.
- The command exits non-zero if recall is below `--min-recall` or any case errored, so it can gate a release.
- The output lists, for every case, which expected policies were **missed** and which **extra** ones were flagged.

Run evaluation against a **test database** loaded with the historical policy versions, never the live one.

Finding why something was missed

For each missed policy, check where it was lost:

```
-- Did any of its clauses reach the judge?
SELECT candidate_number, found_by, stage, result->>'impacted', result->>'why'
FROM judgements WHERE run_id = '<run>' AND candidate_policy = '<missed id>';
```

What you see	Cause	Fix
No rows	It never became a candidate	Check that it's in the group; add explicit references (`--llm-refs`); raise `SEMANTIC_K` / `RERANK_KEEP`; try another embedding model
Rows, all "no"	The judge decided wrongly	Read the pair. Adjust the judge prompt in `app/prompts.py`; turn on escalation; later, fine-tune
Rows, "unsure" turned "no" by escalation	The larger model overruled	Compare both verdicts; decide which model to trust for this kind of change
The changed section wasn't detected	Parsing or diff problem	Check the file's numbering (how-to.md)

Comparing models or settings

Change **one thing at a time** and run the same case set. Record the results in a table like this:

Date	Judge	Escalation	Embed / rerank	K / keep	Recall	Precision	Maybe rate	Time per run

Small case sets are noisy. A difference of one case in 15 is about 7 points of recall. Don't adopt a change on a gain smaller than that unless it holds on the hold-out set.

After launch

- Every reviewer decision on a Maybe is a new labelled example. They are collected through the review queue; see review-and-training.md.
- Each quarter, rerun the evaluation and add new real changes to the case set.
- Before fine-tuning, keep the hold-out cases out of the training data.